SERVIÇO NACIONAL DE APRENDIZAGEM INDUSTRIAL – SENAI
DESENVOLVIMENTO DE SISTEMAS

MAYCON SIQUEIRA DE MORAES

DESIGN PATTERN - SINGLETON

NOVA LIMA – MG
2026

MAYCON SIQUEIRA DE MORAES

DESIGN PATTERN - SINGLETON

Projeto de pesquisa apresentado ao Curso Técnico de Desenvolvimento de Sistemas sobre o padrão Singleton.

Orientado por: Instrutor Frederico Martins Aguiar

NOVA LIMA - MG

2026

Resumo

Este projeto de pesquisa apresenta o padrão de projeto criacional **Singleton**, cuja função é garantir que uma classe possua apenas uma única instância durante todo o ciclo de vida da aplicação, provendo um ponto de acesso global e seguro.
Através de uma análise detalhada, o trabalho explora como o Singleton resolve problemas críticos de **controle de recursos**, como conexões com bancos de dados e gerenciamento de logs, evitando conflitos de acesso e sobrecarga de memória. Para ilustrar o conceito de centralização, utiliza-se uma analogia com a personagem **Ningguang** (do jogo *Genshin Impact*), facilitando a compreensão da estrutura de "instância única".

Abstratc

This research project presents the Singleton creational design pattern, which ensures that a class has only a single instance throughout the entire application lifecycle, providing a global and secure access point. Through a detailed analysis, the study explores how Singleton addresses critical resource control issues, such as database connections and log management, preventing access conflicts and memory overhead. To illustrate the concept of centralization, an analogy is drawn with the character Ningguang (from the game *Genshin Impact*), facilitating the understanding of the "single instance" structure.

Sumário de Figuras

Figura 1 – Gang of Four _____ *1*

Figura 2 - Exemplo de Singleton _____ *3*

Figura 3 - Cód Singleton Cadeado _____ *7*

Figura 4 - Cód Singleton Cofre e Chave _____ *7*

Sumário de Tabelas

Tabela 1 - Abordagens Singleton _____ *9*

Sumário

1. Introdução 1
1.1. O que são Padrões Criacionais (Creational Patterns)? 2
1.2. O que é uma Instância? 2
2. Definição do Padrão Singleton 3
3. Problema que resolve 4
3.1. Controle de Instanciação e Recursos 4
3.2. Acesso Global Seguro 4
4. Estrutura 5
5. Participantes 5
6. Justificativa da Escolha 6
6.1. Singleton como Sincronização 6
6.2. Gestão de Serviços Globais 6
7. Explicação da implementação no projeto 7
8. Análise Crítica 8
9. Comparação com abordagens sem o padrão 8
10. Vantagens 10
11. Desvantagens 10
12. Exemplos reais de uso no mercado 11
13. Referências Bibliográficas 12

1. Introdução

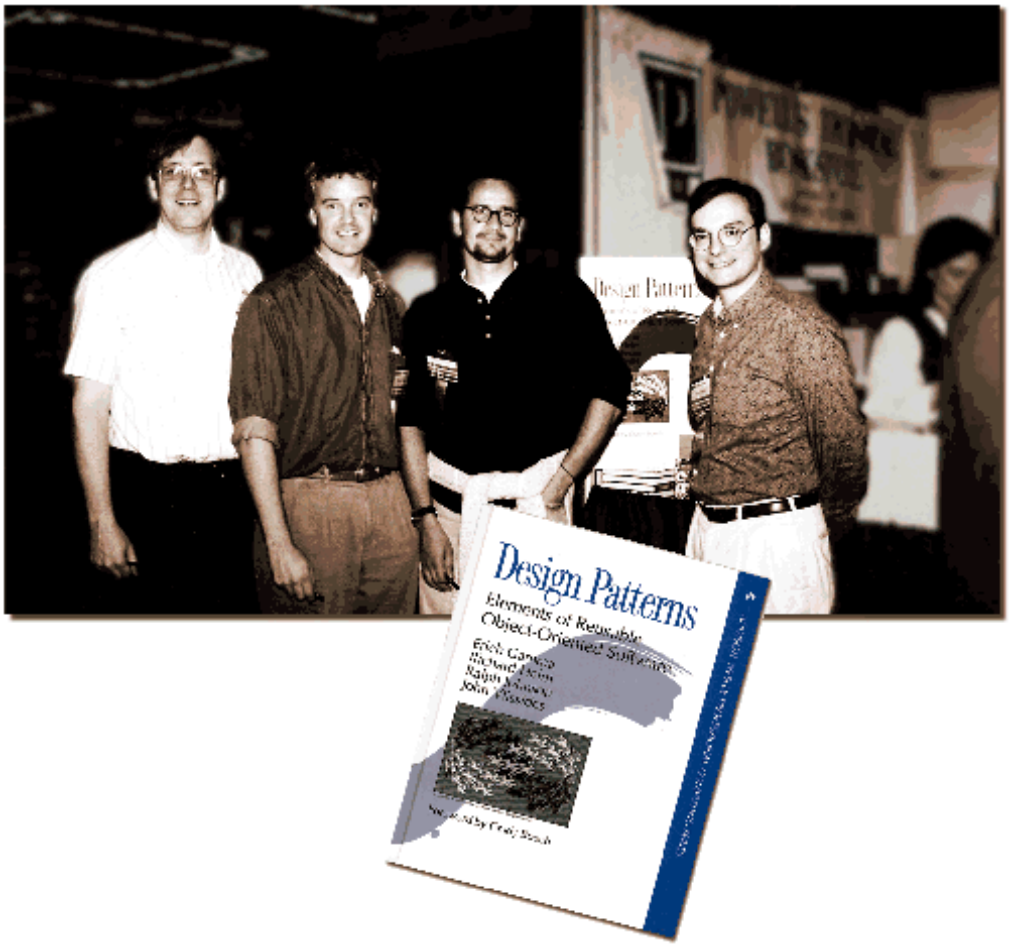
A engenharia de software moderna busca criar sistemas fortes e fáceis de manter. Para isso, os desenvolvedores utilizam **Padrões de Projeto (Design Patterns),** que são soluções comprovadas para problemas que aparecem com frequência no dia a dia.

Essa prática se tornou um padrão na indústria em 1994, graças ao livro ***"Design Patterns: Elements of Reusable Object-Oriented Software***", escrito por quatro autores (Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides) conhecidos como **Gang of Four (GoF).**

Eles catalogaram 23 padrões e os dividiram em três grandes grupos:

1. **Criacionais:** Focam em como os objetos são criados.
2. **Estruturais:** Focam em como classes e objetos são montados para formar estruturas maiores.
3. **Comportamentais:** Focam na comunicação entre os objetos.

Figura 1 – Gang of Four



Fonte: Imagens do Google

1.1. O que são Padrões Criacionais (Creational Patterns)?

O foco desses padrões é simplificar a instanciação (a criação) de objetos. Em vez de o sistema criar objetos de forma direta e rígida, os padrões criacionais funcionam como uma "caixa preta":

- **O que eles fazem:** Eles escondem os detalhes de como um objeto é criado e qual classe específica está sendo usada.
- **A vantagem:** Isso torna o sistema mais flexível. Se você precisar mudar a forma como um objeto é montado ou trocar uma peça do sistema, o restante do código não precisa ser alterado, pois ele não conhece os detalhes internos da criação.

1.2. O que é uma Instância?

É um objeto real e concreto que foi criado na memória do computador a partir de uma Classe.

- A Classe define o "tipo" (ex: Carro).
- A Instância é o objeto específico (ex: Uno Vermelho, Placa ABC-1234).

Por que isso é importante?

Uma única Classe pode gerar infinitas instâncias, e cada uma delas pode ter características diferentes, mesmo seguindo o mesmo molde:

- Classe: Cachorro
- Instância 1: Nome: "Eros", Raça: "Vira-lata".
- Instância 2: Nome: "Bela Princesa", Raça: "Pitbull".

2. Definição do Padrão Singleton

O **Singleton** (também conhecido como **Carta Única)** é um padrão de projeto criacional que garante que uma classe tenha apenas uma instância em todo o ciclo de vida da aplicação, fornecendo um ponto global de acesso a essa instância.
Esse padrão é útil em situações onde é necessário ter exatamente um objeto de uma classe para coordenar ações em todo o sistema.

Figura 2 - Exemplo de Singleton

Fonte: Gemini

Na imagem, utilizei a personagem Ningguang do jogo de Genshin Impact, que é a *Tianquan* da nação de Liyue, ela possui seu palácio chamado Câmara de Jade, onde todos os assuntos da nação passam por ela.
O singleton pode ser descrito como a classe Câmara de Jade, tendo a instância única Tianquan: Ningguang, onde todo o sistema é coordenado em um único lugar.

3. Problema que resolve

Diferente de classes comuns, o Singleton assume um papel duplo:

1. Ele executa suas tarefas (como salvar um log ou conectar ao banco),
2. E, ao mesmo tempo, gerencia seu próprio ciclo de vida.

Ele decide quando nascer e garante que ninguém mais crie "clones" dele, evitando que o sistema fique sobrecarregado com objetos desnecessários.

3.1. Controle de Instanciação e Recursos

Alguns recursos do computador são caros ou limitados. O Singleton impede o caos em dois cenários críticos:

- **Conexões de Banco de Dados:** Criar uma conexão toda vez que um usuário clica em algo é lento e pode derrubar o servidor. O Singleton garante que todos usem a mesma conexão (ou um pool controlado).
- **Acesso a Arquivos (Logs):** Se duas partes do programa tentarem escrever no mesmo arquivo de log ao mesmo tempo, o Windows ou Linux vai travar a operação. O Singleton centraliza o acesso, garantindo que apenas um "braço" escreva por vez, evitando erros fatais.

3.2. Acesso Global Seguro

Antigamente, usavam-se variáveis globais, mas elas eram perigosas: qualquer parte do código podia apagá-las ou alterá-las sem querer. O Singleton oferece a mesma facilidade (você o acessa de qualquer lugar), mas com uma camada de proteção:

- **Encapsulamento:** Você pode usar as funções dele, mas não pode deletar a instância ou substituí-la por algo vazio.
- **Segurança:** A instância física fica escondida e protegida, exposta apenas por um canal de "somente leitura" controlado.

4. Estrutura

Para que uma classe se torne um Singleton, ela precisa seguir estas regras:

- **Construtor Privado (private):** Ao tornar o construtor privado, você impede que qualquer outra parte do código use o comando new Singleton(). Só a própria classe pode criar a si mesma.
- **Classe Selada (sealed):** Isso evita que outras classes herdem dela. Se alguém pudesse herdar do Singleton, poderia acabar criando instâncias extras sem querer, quebrando a regra de "uma única cópia".
- **O "Cofre" (Campo Estático):** A classe guarda uma variável privada e estática (ex: _instance) que armazena a única cópia do objeto na memória enquanto o programa estiver rodando.
- **O Portal de Acesso (Propriedade Instance):** Como ninguém pode dar *new*, a classe oferece uma propriedade pública (geralmente chamada de Instance).
- **Lazy Initialization (Inicialização Preguiçosa):** A primeira vez que você pede a instância, a classe a cria. Nas vezes seguintes, ela apenas entrega a que já está pronta.

5. Participantes

O Participante "Cliente"

O Cliente é qualquer parte do seu código que precise usar o Singleton (um serviço de log, uma configuração de banco de dados, etc.).

- Ele é passivo: não tenta criar o objeto.
- Ele apenas "chama" o Singleton quando precisa, usando algo como: *Configuracao.Instance.Salvar()*.

6. Justificativa da Escolha

Em aplicações modernas, o WPF utiliza uma janela principal (*MainWindow*) que troca dinamicamente seu conteúdo (*View*) conforme a *ViewModel* atual muda. Para isso, utiliza-se um *NavigationStore* (Repositório de Navegação).

6.1. Singleton como Sincronização

A aplicação do Singleton no *NavigationStore* resolve o problema de dessincronização de instâncias:

- **O Problema:** Se diferentes partes do sistema criarem novas instâncias do repositório de navegação, o *Data Binding* (vinculação de dados) da Janela Principal pode "quebrar", pois ela estaria ouvindo um objeto, enquanto o comando de navegação altera outro.
- **A Solução:** O Singleton garante uma única instância na memória. Assim, quando qualquer botão altera a *CurrentViewModel*, a *MainWindow* reage instantaneamente, pois ambas referenciam o mesmo objeto.

6.2. Gestão de Serviços Globais

O padrão também é aplicado em serviços que precisam ser acessados por todo o software sem a necessidade de passar referências complexas via construtores:

- **SessionManager:** Centraliza os dados do usuário logado (tokens e perfis) de forma protegida.
- **Logging:** Centraliza a escrita de logs no sistema de arquivos, evitando conflitos de acesso simultâneo (*file locking*) que poderiam travar a aplicação.

OBS:

- **File Locking:** Mecanismo que impede que múltiplos usuários ou processos editem um arquivo simultaneamente, garantindo a integridade dos dados e evitando conflitos de versão

7. Explicação da implementação no projeto

A Câmara de Jade possui apenas uma liderança (a Tianquan, Ningguang) e um cofre centralizado onde todos os fundos de Mora (moeda oficial de Teyvat) são contabilizados.
O padrão Singleton atua como uma sincronização obrigatória. Ele garante que, não importa quantas vezes o sistema solicite acesso à sessão para coletar taxas ou ler o saldo, será entregue sempre a mesma e única instância.

Figura 3 - Cód Singleton Cadeado

```
private GerenciadorSessao()
{
    MoraAcumulada = 50000000;
}
```

Fonte: Jet Brains Rider

A linha *private GerenciadorSessao()* funciona como um **Cadeado**, bloqueando a criação externa de objetos. Ninguém de fora da própria classe consegue usar o comando de inicialização padrão.

Figura 4 - Cód Singleton Cofre e Chave

```
public class GerenciadorSessao : INotifyPropertyChanged
{
    private static readonly Lazy<GerenciadorSessao> _instance =
        new Lazy<GerenciadorSessao>(() => new GerenciadorSessao());

    1 usage   May Croft
    public static GerenciadorSessao Instance => _instance.Value;
```

Fonte: Jet Brains Rider

O campo private *static readonly Lazy _instance* funciona como um **Cofre**. Esta é a própria classe guardando uma cópia única (e blindada) dela mesma em um espaço isolado de memória.
A propriedade *public static GerenciadorSessao Instance* funcionará como uma **Chave**. Como as outras classes não podem criar uma sessão nova, elas são obrigadas a pedir acesso passando por esse portal, garantindo a centralização.

8. Análise Crítica

A aplicação do padrão Singleton deve ser uma decisão consciente e estratégica, e não um hábito automático do desenvolvedor.

Os pontos principais são:

- **Ponderação de Trade-offs:** O papel do desenvolvedor não é apenas aplicar padrões, mas avaliar se os benefícios de uma solução superam seus custos técnicos e operacionais.
- **Contra a "Programação Orientada a Padrões":** Critica-se a tendência de implementar padrões apenas por formalidade sintática, ignorando o contexto e a real necessidade do projeto.
- **Foco na Manutenibilidade:** O código deve ser escrito pensando na facilidade de manutenção futura. Um Singleton mal aplicado pode se tornar um obstáculo para a evolução do sistema, independentemente de quão "correta" pareça sua implementação inicial.

9. Comparação com abordagens sem o padrão

A escolha entre essas estruturas não é apenas estética, mas impacta diretamente a manutenção e a performance do software:

- **Classes Estáticas:** São ideais para funções utilitárias sem estado (como cálculos matemáticos). Contudo, são "engessadas": não aceitam herança ou interfaces, dificultam testes unitários e pesam no carregamento inicial da aplicação.
- **Padrão Singleton:** Uma evolução sobre as estáticas. Permite o uso de interfaces e o carregamento sob demanda (*Lazy Loading*), o que economiza memória. No entanto, ainda sofre com o acoplamento forte e a "dependência oculta", dificultando rastrear quem usa o quê.
- **Injeção de Dependência (DI):** É um padrão de projeto que reduz o acoplamento entre classes, transferindo a responsabilidade de criar dependências para um componente externo (*container*), em vez de a própria classe instanciá-las, facilitando a manutenção, reutilização de código e testes unitários.

Tabela 1 - Abordagens Singleton

Aspecto	Classe Estática	Padrão Singleton	Injeção de Dependência (DI)
Polimorfismo	Não permite	Permite (Interfaces/Herança)	Totalmente baseado em Interfaces
Carregamento	Rígido (Framework)	Flexível (Lazy Loading)	Gerenciado pelo Container (IoC)
Acoplamento	Altíssimo (Rígido)	Alto (Dependência oculta)	Baixo (Loose Coupling)
Testabilidade	Muito Difícil	Complexa/Instável	Excelente (Uso de Mocks)

Embora o Singleton (especialmente *System.Lazy<-T->*) ainda tenha utilidade em cenários específicos ou sistemas legados, a Injeção de Dependência é a escolha ideal na arquitetura atual de .NET.

Ela resolve o problema da "dependência oculta", transformando o que seria um acesso global opaco em uma necessidade clara e transparente no construtor das classes, promovendo um código mais modular, limpo e testável.

OBS:

- **Depêndencia Oculta:** É quando um componente depende de recursos externos (classes, serviços ou variáveis globais) não explicitos na sua interface ou construtor. Elas dificultam a manutenção e testabilidade, pois o código funciona de forma não óbvia, exigindo análise interna para entender o acoplamento.

10. Vantagens

- **Eficiência de Recursos:** Evita a criação e destruição constante de instâncias, mantendo a ocupação da memória previsível.
- **Performance Instantânea:** Maximiza o tempo de resposta após a primeira inicialização, garantindo fluidez em fluxos de alta frequência.
- **Inicialização Preguiçosa (Lazy Loading):** Através de Lazy<-T->, evita o "choque térmico" no arranque do software (como o cold start em interfaces WPF), adiando operações pesadas (como conexões SQL) para o momento exato do uso.
- **Acessibilidade Global Simplificada:** Oferece um ponto de acesso direto em árvores complexas de UI, evitando a necessidade de "comilões de construtores" (injeção manual excessiva de componentes pesados em cada camada).

11. Desvantagens

- **Insegurança:** A instância única expõe o encapsulamento a modificações incontroláveis. Em ambientes multithread (multi-tarefas), isso pode gerar erros "fantasmas" se não houver sincronização rigorosa.
- **Falta de Transparência:** Como o Singleton é chamado diretamente no corpo dos métodos (ex: Logger.Instance.Execute()), ele não aparece no construtor da classe. Impedindo o desenvolvedor de identificar as necessidades da classe em uma API pública.
- **Código "Espaguete" Frágil:** A interdependência oculta dificulta a manutenção a longo prazo, transformando o projeto em um campo minado onde mudanças em uma ponta quebram funcionalidades.
- **Dificuldade em Testes Unitários:** O Singleton retém estado entre os testes no ambiente de execução. Isso causa os chamados Flaky Tests (testes que falham aleatoriamente), pois um teste pode "envenenar" o resultado do próximo, exigindo recursos técnicos (como Reflection) para resetar a instância.

12. Exemplos reais de uso no mercado

O Singleton não é apenas sobre "ter uma instância", mas sobre impor ordem nos sistemas distribuídos que competem por recursos limitados e sensíveis.

1. **Sincronização de Logs e Auditoria**

Centraliza o fluxo de escrita em sistemas como *Splunk* ou *Serilog*.

- **O Problema:** Vários módulos tentando escrever no mesmo arquivo ou banco de dados ao mesmo tempo geram erros de I/O (file locks) e corrupção de dados.
- **A Solução:** O Singleton garante uma fila única e thread-safe, organizando as mensagens de forma sequencial antes da persistência física no disco ou nuvem.

2. **Cache de Configurações e Metadados**

Funciona como uma memória de leitura rápida para parâmetros globais (chaves de API, variáveis de ambiente).

- **O Problema:** Ler arquivos do disco repetidamente gera latência e consome processamento desnecessário.
- **A Solução:** O Singleton carrega esses dados uma única vez na inicialização, oferecendo acesso instantâneo em memória para todo o ecossistema do software.

3. **Controle de Hardware e Spoolers**

Gerencia o acesso a componentes físicos ou drivers de baixo nível (impressoras, leitores, catracas).

- **O Problema:** Hardwares são, por natureza, exclusivos. Dois processos tentando enviar comandos simultâneos podem causar perda de pacotes ou travamento de drivers.
- **A Solução:** O padrão encapsula o acesso ao driver, garantindo que apenas um comando por vez seja processado através de um buffer controlado.

13. Referências Bibliográficas

WANG, Ligang et al. Report of Design Patterns.
Disponível em:
<https://ecs.syr.edu/faculty/fawcett/handouts/cse776/Lecture20/References/Design_patterns_Report.pdf>. Acesso em: 21 mar. 2026.

Implementing the singleton pattern in C#.
Disponível em: <https://csharpindepth.com/articles/singleton>. Acesso em: 21 mar. 2026.

KURMI, Vivek. Singleton Design Pattern - Step by step Guide.
Disponível em: <https://dev.to/kurmivivek295/singleton-design-pattern-step-by-step-guide-2bbb>. Acesso em: 21 mar. 2026.

The Singleton pattern in C# today is not your dad's one!
Disponível em: <https://blog.postsharp.net/singleton-pattern>.
Acesso em: 21 mar. 2026.

What are the real world applications of the singleton pattern?
Disponível em: <https://stackoverflow.com/questions/733842/what-are-the-real-world-applications-of-the-singleton-pattern>. Acesso em: 22 mar. 2026.

NARAPAREDDY, Shyamprasad. Singleton pattern: Exploring the Best Use Cases for C# Developers.
Disponível em: <https://nshyamprasad.medium.com/singleton-pattern-exploring-the-best-use-cases-for-c-developers-5e949fc7e4eb>. Acesso em: 22 mar. 2026.

RAHMAN, Hasinur. Singleton Design Pattern Explained with Real-life examples.
Disponível em: <https://medium.com/@hasinur1997/singleton-design-pattern-explained-with-real-life-examples-d020495c4d82>. Acesso em: 22 mar. 2026.

MÜLLER, Martin. Singleton Pattern Disadvantages and solutions.
Disponível em: <https://medium.com/@cadothek/singleton-pattern-disadvantages-and-solutions-20b99d9e1986>. Acesso em: 22 mar. 2026.

JANINDUMALEESHA. The singleton pattern in .NET: A practical guide for developers.
Disponível em: <https://medium.com/@janindumaleesha99/the-singleton-pattern-in-net-a-practical-guide-for-developers-140d3c3464fd>. Acesso em: 22 mar. 2026.

BILLWAGNER. Lazy initialization.
Disponível em: <https://learn.microsoft.com/en-us/dotnet/framework/performance/lazy-initialization>. Acesso em: 22 mar. 2026.

TAYAL, Dhruv. Mastering the Singleton Design Pattern with Lazy Initialization in C#.
Disponível em: <https://dhruv-tayal.medium.com/mastering-the-singleton-design-pattern-with-lazy-initialization-in-c-fa8ae052f5d6>. Acesso em: 22 mar. 2026.

Why you should prefer singleton pattern over a static class?
Disponível em: <https://volosoft.com/blog/Prefer-Singleton-Pattern-over-Static-Class>. Acesso em: 22 mar. 2026.

VEIGA, Leandro. Mastering C# design patterns: Practical examples of singleton, factory, and observer.
Disponível em: <https://dev.to/leandroveiga/mastering-c-design-patterns-practical-examples-of-singleton-factory-and-observer-36he>. Acesso em: 22 mar. 2026.